Programko – Roblox a LUA I

Programujeme hry v Roblox Studiu

Učebnica pre krúžok programovania, 5. – 9. ročník ZŠ

Ako čítať túto učebnicu

Časť A (lekcie 1 – 18) je jadro kurzu. Prejdeme ju spolu na krúžku počas **16 stretnutí** – rozpis nájdeš v Prílohe C.

Časť B (lekcie 19 – 23) sú bonusové lekcie. Môžeš si ich spraviť doma alebo ich preberieme v pokračovaní kurzu.

Úlohy označené číslom (napr. 7.3) sú povinné – tie stíha každý.

Bonus pre rýchlych je pre tých, ktorí sú skôr hotoví. Nikto ho nemusí spraviť.

Príloha A je ťahák s najčastejšími príkazmi.

Obsah

Úvod – Než začneme 6

Čo je Roblox 6

Čo potrebuješ 6

Pravidlo krúžku o umelej inteligencii 6

1. Lekcia – Registrácia a inštalácia Roblox Studia 7

Inštalácia Roblox Studia 7

Vytvorenie účtu (ak ho ešte nemáš) 7

2. Lekcia – Úvod do Roblox Studia a jeho spustenie 8

Vytvorenie nového projektu 8

Nastavenie Exploreru a Properties 8

3. Lekcia – Pridávanie objektov do hry a testovanie 9

Typy objektov 9

Testovanie a zastavenie hry 9

4. Lekcia – Práca s objektami 10

Možnosti práce s objektami 10

5. Lekcia – Konzola a príkazy print() a task.wait() 12

Nastavenie Output a Command Baru 12

Prvý príkaz print() 12

Komentáre 12

Príkaz task.wait() 12

Viac hodnôt v jednom print() 12

6. Lekcia – Pridávanie skriptov 14

Pridávanie skriptov 14

Spustenie skriptu 14

7. Lekcia – Nastavovanie vlastností objektov 16

Nastavovanie vlastností 16

Príklady nastavovania vlastností 16

8. Lekcia – Keď to nefunguje: čítame chyby 18

Kde chyby uvidíš 18

Ako sa číta chybová hláška 18

Päť najčastejších chýb 18

Ladenie pomocou print() 19

9. Lekcia – Premenné 21
Čo je premenná 21
Typy premenných 21
Pomenovanie premenných 22
Matematické operácie 22
Zvýšenie hodnoty premennej 22
Spájanie textu 22
10. Lekcia – Funkcie 24
Vysvetlenie funkcií 24
Vstupný argument vo funkcii 24
Návratová (return) hodnota funkcie 24
11. Lekcia – Podmienky 27
Vetvenie programu pomocou podmienok 27
Porovnávacie operátory 27
Logické operátory 27
Viac možností: elseif 28
12. Lekcia – Mini-projekt: Prekážková dráha 30
Zadanie 30
Povinné prvky 30
Ako na to – postup po krokoch 30
Nápady na prekážky 30
13. Lekcia – ScreenGui a MouseButton1Click 32
ScreenGui 32
LocalScript – a prečo nie obyčajný Script 32
Vlastnosť MouseButton1Click 33
14. Lekcia – Udalosť Touched 34
Vysvetlenie udalosti 34
Čo je hit 34
15. Lekcia – Spracovanie vstupu od hráča 36
Vysvetlenie spracovania vstupu 36
Zistenie, na ktorý objekt hráč klikol 36
16. Lekcia – For cyklus 38
Použitie for cyklu 38

For cyklus pre časti modelu ... 38
17. Lekcia – While cyklus ... 40
Použitie while cyklu ... 40
Nekonečný cyklus ... 40
18. Lekcia – Záverečný projekt ... 42
Zadanie ... 42
Úrovne – vyber si, na čo si trúfaš ... 42
Postup ... 42
Zdieľanie hry so spolužiakmi ... 42
Otázky pri testovaní hry spolužiaka ... 43
Prezentácia (2 minúty) ... 43
Časť B – Bonusové lekcie ... 44
19. Lekcia – Prehrávanie zvuku ... 45
Pridanie zvuku ... 45
Prehrávanie zo skriptu ... 45
20. Lekcia – Pridanie NPC postavy ... 47
Ako pridať do hry NPC ... 47
Pridanie interaktívneho okna ... 47
Naprogramovanie funkcionality ... 47
21. Lekcia – Nástroje (Tools) ... 50
Čo je nástroj ... 50
Pridanie nástroja z Toolboxu ... 50
Pridanie nástroja programom ... 50
Vytvorenie vlastného nástroja ... 50
22. Lekcia – Udalosti Equipped a Unequipped ... 52
Použitie udalostí ... 52
23. Lekcia – Animácie ... 54
Vytvorenie animácie ... 54
Pridanie animácie do hry ... 54
Programovanie prehratia animácie ... 55
ProximityPrompt a animácie ... 55
Príloha A – Ťahák ... 57
Základné príkazy ... 57

Premenné 57
Vlastnosti objektov 57
Podmienky 57
Funkcie 58
Cykly 58
Udalosti 58
Užitočné materiály a farby 58
Príloha B – Časté chyby a ako ich odstrániť 59
Chybové hlášky 59
Nič sa nedeje a v Outpute nie je chyba 59
Objekt padá alebo sa nedá naň stúpiť 59
Hra zamrzla a nedá sa vypnúť 59
Zvuk sa neprehrá 59

Úvod – Než začneme

Čo je Roblox

Roblox je herná platforma, na ktorej sa dajú hry hrať, ale aj vytvárať a programovať. My budeme používať **Roblox Studio** – program, v ktorom hry vznikajú.

Programovať budeme v jazyku **Lua**. Je to skutočný programovací jazyk, ktorý sa používa aj mimo Robloxu. Všetko, čo sa naučíš – premenné, podmienky, cykly, funkcie – funguje rovnako v desiatkach ďalších jazykov.

Čo potrebuješ

- Počítač alebo notebook, na ktorý sa dá nainštalovať program
- Pripojenie na internet
- Roblox konto (vytvoríme si ho v 1. lekcii)
- Trpezlivosť. Prvý program nikdy nefunguje na prvýkrát – ani programátorom.

Pravidlo krúžku o umelej inteligencii

Roblox Studio má v sebe **AI pomocníka (Assistant)**, ktorý vie kód napísať za teba. V prvej časti kurzu ho **nepoužívame**.

Nie preto, že by bol zlý. Ale ak ti kód napíše niekto iný, nenaučíš sa ho čítať – a keď potom niečo prestane fungovať, nebudeš vedieť prečo. Najprv sa nauč programovať sám. Na záverečnom projekte si o AI povieme viac a vtedy ju už použiješ vedome.

Čo ťa čaká

- Naučíš sa stavať a tvarovať objekty v hernom svete.
- Naučíš sa písať skripty, ktoré tie objekty rozhýbu.
- Postavíš si vlastnú hru a dáš ju zahrať spolužiakom.

1. Lekcia – Registrácia a inštalácia Roblox Studia

Roblox Studio budeme používať po celý čas. V tejto lekcii si ho nainštalujeme, vytvoríme konto a zoznámime sa s ním.

Inštalácia Roblox Studia

- Choď na stránku **https://www.roblox.com/create**
- Klikni na tlačidlo **Start creating**
- Klikni na **Download Studio**
- Vyskočí okno – ulož súbor (**Save file**)
- Počkaj, kým sa súbor stiahne, a potom ho otvor
- Počkaj, kým sa Roblox Studio nainštaluje

Vytvorenie účtu (ak ho ešte nemáš)

- Choď na stránku **https://www.roblox.com/**
- Vyplň formulár
- Klikni na **Sign up**
- Zodpovedaj verifikačné otázky

Heslo a bezpečnosť

Použi **dostatočne silné heslo**, ale také, aby si si ho zapamätal. Je vhodné si ho niekde zapísať.

Heslo **nikdy nikomu nedávaj** – ani spolužiakovi, ani nikomu, kto ti napíše v hre. Nikto z Robloxu ťa o heslo nikdy nepožiada.

Pri registrácii **zadaj svoj skutočný dátum narodenia**. Roblox podľa neho nastaví bezpečnostné obmedzenia, ktoré ťa chránia.

1.1 Úloha – Je to nainštalované a som prihlásený

Uisti sa, že inštalácia prebehla úspešne do tvojho počítača alebo notebooku.

Vytvor si účet a prihlás sa do Roblox Studia.

Čo si sa naučil

- Roblox Studio je program, v ktorom sa hry vytvárajú.
- Heslo patrí len tebe.

2. Lekcia – Úvod do Roblox Studia a jeho spustenie

Vytvorenie nového projektu

Otvor Roblox Studio a vyber si **template** (šablónu). Ak by si sa neskôr chcel dostať naspäť na úvodné okno so šablónami, klikni na **File → Open from Roblox**.

Odporúčame používať **prvé tri šablóny**, ktoré neobsahujú ďalšie objekty – bude sa nám s nimi ľahko pracovať.

Nastavenie Exploreru a Properties

Je dôležité, aby sme videli dve okná – **Explorer** a **Properties**. Sú základom celej našej tvorby.

- **Explorer** je zoznam všetkého, čo sa v hre nachádza. Je to strom – objekty môžu byť vnorené do seba.
- **Properties** ukazuje vlastnosti práve vybraného objektu (farbu, veľkosť, pozíciu a ďalšie).

Ak ich nevidíš, klikni na **View** a potom na **Explorer** a **Properties**. Vpravo by si ich už mal mať k dispozícii.

2.1 Úloha – Nastav si pracovnú plochu

Vytvor nový projekt z prvej šablóny.

Zapni si okná **Explorer** a **Properties**.

Klikni v Exploreri na objekt `Baseplate` a pozri sa, aké vlastnosti sa objavili v okne Properties. Nájdi vlastnosť `Color`.

Čo si sa naučil

- **Explorer** = zoznam všetkého v hre.
- **Properties** = vlastnosti vybraného objektu.
- Obe okná zapneš cez menu **View**.

3. Lekcia – Pridávanie objektov do hry a testovanie

Typy objektov

V Roblox Studiu máme na výber viacero typov **Partov** (objektov): **Block**, **Sphere**, **Wedge**, **Cylinder** a **Corner Wedge**.

Objekt pridáš tak, že klikneš na malú šípku pri tlačidle **Part** – zobrazia sa ti možnosti. Po vybraní jednej z nich sa objekt vloží do hry a hneď ho uvidíš.

Testovanie a zastavenie hry

Hru zapíname kliknutím na **Play** (modrý trojuholník) a vypíname kliknutím na **Stop** (červený štvorec).

Odporúčame používať aj tlačidlo **Play Here**, ktoré vloží tvoju postavu presne tam, kde máš momentálne nastavenú kameru. Ušetrí ti to dlhé behanie po mape.

Testovanie sa dá spúšťať v sekcii **Home** aj v sekcii **Test** – používaj ten spôsob, ktorý sa ti páči najviac.

Zmeny počas hry sa neuložia

Ak zmeníš hru počas toho, ako je zapnutá (beží Play), **zmena sa neuloží**. Po stlačení Stop sa všetko vráti do pôvodného stavu.

Toto je najčastejšia frustrácia začiatočníkov: „Ja som to predsa spravil!“ Spravil – ale počas behu hry. Vždy sa najprv presvedč, že hra **nebeží**.

3.1 Úloha – Pridávame prvé objekty

Pridaj do hry jeden kváder (Block), jednu guľu (Sphere) a ešte jeden ľubovoľný objekt.

3.2 Úloha – Prvý štart

Spusti hru pomocou **Play** a vyskúšaj si pohyb postavičky. Následne hru zastav tlačidlom **Stop**.

Spusti hru druhýkrát pomocou **Play Here** a potom ju zastav. Všimni si rozdiel, kde sa tvoja postava objavila.

3.3 Bonus – Prvá stavba

Postav z objektov jednoduchý most alebo schody, po ktorých sa dá prejsť. Vyskúšaj, či po nich tvoja postava naozaj vyjde.

Čo si sa naučil

- Objekty (Party) sa pridávajú cez šípku pri tlačidle Part.
- Play / Stop / Play Here.
- Počas behu hry sa zmeny neukladajú.

4. Lekcia – Práca s objektami

Možnosti práce s objektami

S objektmi vieme narábať štyrmi základnými nástrojmi. Nájdeš ich v sekcii **Home** alebo v sekcii **Model**.

Nástroj	Čo robí
Select	Vyberie objekt, s ktorým chceš pracovať
Move	Posúva objekt do všetkých smerov
Scale	Mení veľkosť a tvar objektu
Rotate	Otáča objekt

Select

Nástroj, ktorým si objekt vyberieš. Vybraný objekt sa zvýrazní a jeho vlastnosti sa zobrazia v okne Properties.

Move

Nástroj, ktorým môžeš objekt ľubovoľne posúvať do všetkých smerov.

Scale

Táto funkcia je veľmi dôležitá a budeme ju používať často. Vďaka nej vieme objekt tvarovať, zväčšovať a zmenšovať.

Rotate

S týmto nástrojom vieme objekt otáčať do rôznych strán.

Drž a ťahaj

Pri nástrojoch Move a Scale ťahaj za **farebné šípky alebo guličky** – každá farba je jeden smer.

Ak chceš objekt posúvať plynulo (nie po skokoch), vypni v sekcii Model prichytávanie **Snap to Grid**.

4.1 Úloha – Hráme sa s objektom

Na objekte si vyskúšaj všetky štyri nástroje.

Presuň objekt (**Move**), potom ho skús zväčšiť a zmenšiť (**Scale**). Na záver mu daj miernu rotáciu (**Rotate**).

4.2 Bonus – Moje ihrisko

Postav si malé ihrisko: plošinu, na ktorú sa dá vyjsť, a aspoň tri prekážky rôzneho tvaru a veľkosti. Vyskúšaj to v hre.

Čo si sa naučil

- Select, Move, Scale, Rotate – štyri základné nástroje.
- Scale budeš používať najčastejšie.

5. Lekcia – Konzola a príkazy print() a task.wait()

Nastavenie Output a Command Baru

Konzola je veľmi dôležitý prvok. Poskytuje nám na výstupe všetky informácie, ktoré potrebujeme – informuje nás o chybách a je to náš programátorský výstup pri použití príkazu `print()`.

Aby sme tento výstup videli, uisti sa, že v sekcii **View** máš zakliknuté **Output**. Na písanie prvého príkazu potrebujeme aj príkazový riadok, preto zaklikni aj **Command Bar**.

Prvý príkaz print()

Príkaz `print()` zobrazí na výstupe to, čo napíšeme do zátvoriek.

```
print("Ahoj")   -- vypíše na výstup slovo Ahoj
```

Komentáre

Všimol si si tie dve pomlčky `--`? Všetko, čo je za nimi, je **komentár**. Počítač to ignoruje – je to poznámka pre človeka, ktorý kód číta.

```
-- Toto je komentár. Počítač ho preskočí.
print("Ahoj")   -- komentár môže byť aj na konci riadku
```

Komentáre používaj. Keď sa k svojmu kódu vrátiš o dva týždne, budeš rád, že si si tam napísal, čo to robí.

Príkaz task.wait()

Príkaz `task.wait()` znamená **čakanie**. Do zátvorky píšeme počet sekúnd.

```
task.wait(3)   -- počká 3 sekundy
```

Nie wait(), ale task.wait()

V starších návodoch na internete uvidíš len `wait(3)`. Roblox tento príkaz **nahradil** novším `task.wait(3)`.

Starý spôsob zatiaľ funguje, ale je zastaraný a Roblox ho postupne ruší. My budeme od začiatku používať **`task.wait()`**, aby si sa nemusel nič preúčať.

Viac hodnôt v jednom print()

Do príkazu `print()` môžeš dať aj viac vecí naraz – oddelíš ich čiarkou.

```
print("Mám rokov:", 12)   -- vypíše: Mám rokov: 12
```

5.1 Úloha – Pozdrav z konzoly

Do Command Baru napíš príkaz, ktorý na konzolu vypíše tvoje meno.

5.2 Úloha – Počítame v konzole

Do Command Baru napíš príkaz, ktorý vypíše výsledok príkladu 15 + 27. Čísla píš priamo do zátvorky, bez úvodzoviek.

Nápoveda: `print(15 + 27)`

5.3 Bonus – Veta aj s číslom

Napíš jeden príkaz `print()`, ktorý vypíše vetu `Mám rokov: 12` (s tvojím vekom). Použi čiarku.

Čo si sa naučil

- `print()` vypisuje na výstup do okna **Output**.
- `task.wait(n)` počká n sekúnd.
- `--` označuje komentár, ktorý počítač ignoruje.

6. Lekcia – Pridávanie skriptov

Pridávanie skriptov

Ak pôjdeš myšou na nejaký objekt v Exploreri, objaví sa **plusko „+“**.

Klikni na „+“ a z ponuky vyber **Script** (šedá ikonka zápisníka).

Práve sme k objektu pripojili skript, kde môžeme programovať jeho správanie. Zobrazil sa nám textový súbor s automaticky prednastaveným príkazom. Sme pripravení programovať.

Spustenie skriptu

Zmeň pôvodný skript na nasledujúci a spusti hru. Sledujme spoločne, čo sa deje:

```
task.wait(3)
print("Hello world!")
```

Po spustení hry sa text v Outpute zobrazí až po 3 sekundách vďaka príkazu `task.wait()`. Príkaz počká toľko sekúnd, koľko je v zátvorke, a potom sa vypíše na výstup.

Command Bar verzus skript

Command Bar vykoná príkaz **hneď**, ako stlačíš Enter. Je to na rýchle vyskúšanie.

Skript sa vykoná až vtedy, keď **spustíš hru** tlačidlom Play. To je ten skutočný program.

6.1 Úloha – Programátorský pozdrav

Po 2 sekundách od spustenia hry nech sa zobrazí `Ahoj` a po ďalších 2 sekundách vypíš, ako sa voláš.

6.2 Úloha – Budíček

Vypisuj postupne pomocou príkazov `print()` a `task.wait()` odpočítavanie sekúnd `3`... `2`... `1`... a nakoniec `crrn crrn`.

6.3 Bonus – Okomentuj svoj kód

Vráť sa k úlohe 6.2 a ku každému riadku dopíš komentár, ktorý vysvetľuje, čo ten riadok robí.

Čo si sa naučil

- Skript sa pridáva k objektu cez „+“ v Exploreri.
- Skript beží až po spustení hry.
- Príkazy sa vykonávajú **postupne, riadok po riadku**, zhora nadol.

7. Lekcia – Nastavovanie vlastností objektov

Nastavovanie vlastností

Každý objekt má niekoľko vlastností – napríklad farbu, materiál alebo pozíciu. Tieto vlastnosti môžeme nastavovať priamo v editore v okne **Properties**, alebo pomocou skriptu.

V skripte na to máme dva spôsoby:

```
script.Parent.<názov vlastnosti> = <nová hodnota>

workspace.<názov objektu>.<názov vlastnosti> = <nová hodnota>
```

V prvej možnosti sme použili **relatívnu cestu** – `script.Parent` je objekt, ku ktorému je daný skript pripojený. V druhej možnosti sme použili **absolútnu cestu** – objekt hľadáme podľa mena od začiatku.

Príklady nastavovania vlastností

```
workspace.Kvader.Color = Color3.new(0, 0, 1)             -- farba
workspace.Kvader.Material = Enum.Material.Brick          -- materiál
workspace.Kvader.Anchored = true                         -- ukotvenie
workspace.Kvader.CanCollide = false                      -- dá sa cez neho prejsť
workspace.Kvader.Transparency = 1                        -- priehľadnosť
workspace.Kvader.Position = Vector3.new(76.2, 5, -6.8)   -- zmena pozície
```

Veľké a malé písmená sa rozlišujú

Roblox rozlišuje veľké a malé písmená. `Transparency` funguje, `transparency` nie.

Rovnako musí **presne sedieť názov objektu**. Ak sa objekt v Exploreri volá `Kvader` a ty napíšeš `kvader`, program spadne.

Farby jednoduchšie

`Color3.new()` očakáva čísla od **0 do 1**, čo je nezvyk. `Color3.new(0, 0, 1)` je modrá.

Jednoduchšie sa používa `Color3.fromRGB()`, ktoré berie čísla od **0 do 255** – tak, ako ich poznáš z programov na kreslenie:

```
workspace.Kvader.Color = Color3.fromRGB(255, 0, 0)   -- červená
workspace.Kvader.Color = Color3.fromRGB(0, 255, 0)   -- zelená
```

7.1 Úloha – Farbíme objekt

Po 2 sekundách zmeň farbu objektu na červenú a po ďalších 2 sekundách zmeň farbu na zelenú.

7.2 Úloha – Neón

Po 3 sekundách zmeň materiál objektu na `Neon`. Sleduj, ako sa objekt rozsvieti.

7.3 Úloha – Ukotvená guľa

Po 3 sekundách zapni guli ukotvenie (`Anchored`).

Skús s guľou pohnúť v prvých 3 sekundách a potom po zmene. Popíš rozdiel.

7.4 Úloha – Prechodná stena

Po 3 sekundách vypni kolízie so stenou (`CanCollide`). Pozoruj správanie pred zmenou a po nej.

7.5 Úloha – Záhadné zmiznutie

Po 3 sekundách nastav `Transparency` na hodnotu 1 a vypni kolízie.

Po ďalších 2 sekundách nastav `Transparency` naspäť na 0 a kolízie zapni. Pozoruj a popíš zmeny správania.

7.6 Úloha – Zmena polohy

Po 3 sekundách zmeň polohu objektu. Experimentuj so zmenou polohy aj v iných smeroch.

7.7 Bonus – Strašidelný dom

Naprogramuj objekt, ktorý postupne vystrieda **štyri rôzne vlastnosti** – napríklad farbu, materiál, priehľadnosť a polohu. Medzi zmenami nech vždy chvíľu počká.

Čo si sa naučil

- Vlastnosti sa nastavujú cez `script.Parent.<vlastnosť>` alebo `workspace.<meno>.<vlastnosť>`.
- Roblox rozlišuje **veľké a malé písmená**.
- `Color3.fromRGB()` je na farby pohodlnejšie ako `Color3.new()`.

8. Lekcia – Keď to nefunguje: čítame chyby

> **Prečo táto lekcia**
>
> Programátori netrávia väčšinu času písaním kódu. Trávia ho hľadaním, prečo ten kód nefunguje.
> Toto je najužitočnejšia lekcia v celej učebnici. Keď ju zvládneš, prestaneš sa zasekávať.

Kde chyby uvidíš

Chyby sa vypisujú **červeným textom v okne Output**. Ak sa ti niečo nespráva, ako má, prvá vec, ktorú urobíš, je pozrieť sa do Outputu.

Ako sa číta chybová hláška

Chybová hláška vyzerá napríklad takto:

```
Workspace.Kvader.Script:3: attempt to index nil with 'Color'
```

Rozlož si ju na tri časti:

Časť	Význam
`Workspace.Kvader.Script`	**Kde** je chyba – v ktorom skripte
`:3:`	**Na ktorom riadku**. Toto je najdôležitejšie číslo.
`attempt to index nil...`	**Čo** sa stalo

Vždy začni tým, že si nájdeš ten riadok. Polovica práce je hotová.

Päť najčastejších chýb

1. Preklep v názve objektu

```
workspace.Kvadr.Color = Color3.fromRGB(255, 0, 0)
-- Chyba: attempt to index nil with 'Color'
```

Objekt sa volá `Kvader`, nie `Kvadr`. Slovo **`nil`** znamená „nič“ – Roblox taký objekt nenašiel. Vždy skontroluj názov v Exploreri.

2. Veľké a malé písmená

```
workspace.Kvader.color = Color3.fromRGB(255, 0, 0)
-- Chyba: vlastnosť sa volá Color, nie color
```

3. Chýbajúce end

```
local function pozdrav()
    print("Ahoj")
-- Chyba: Expected 'end'
```

Každé `function`, `if`, `for` a `while` sa musí ukončiť slovom `end`. Ak ich máš viac vnorených do seba, musí byť `end` rovnako veľa.

4. Chýbajúca zátvorka alebo úvodzovka

```
print("Ahoj)
-- Chyba: Unfinished string
```

5. Skript je na zlom mieste

Skript musí byť v **Workspace** (napríklad v objekte). Ak ho omylom vložíš inam, jednoducho sa nespustí a v Outpute nebude nič – ani chyba. Ak sa nedeje vôbec nič, skontroluj v Exploreri, kde skript vlastne je.

Ladenie pomocou print()

Keď program nespadne, ale robí niečo iné, než chceš, použi `print()` ako **kontrolné body**:

```
print("1 - skript sa spustil")
task.wait(3)
print("2 - dočakal som")
workspace.Kvader.Transparency = 1
print("3 - nastavil som priehľadnosť")
```

Spusti hru a pozri sa, po ktorý bod sa program dostal. Tam, kde výpisy skončia, je problém.

Zlaté pravidlá hľadania chýb

- Prečítaj **číslo riadku** v chybovej hláške.
- Skontroluj **názvy** – písmenko po písmenku.
- Spočítaj `end` – koľko ich má byť a koľko ich je.
- Použi `print()` ako kontrolné body.
- Vysvetli problém nahlas spolužiakovi. Veľmi často naň prídeš sám v polovici vety.

8.1 Úloha – Nájdi chybu

V každom úryvku je jedna chyba. Nájdi ju a oprav. Skús najprv **bez** počítača, iba očami.

```
1)  workspace.Gula.Transparency = 1
-- objekt sa v Exploreri volá Gulicka

2)  print("Ahoj"

3)  local function pozdrav()
print("Ahoj")

4)  workspace.Kvader.transparency = 1

5)  task.wait("3")
```

8.2 Úloha – Kontrolné body

Vezmi svoje riešenie úlohy 7.5 a doplň doň tri príkazy `print()` ako kontrolné body. Spusti hru a sleduj Output.

8.3 Bonus – Pokaz a oprav

Vezmi funkčný skript a **zámerne** doň sprav chybu. Daj ho spolužiakovi, nech ju nájde. Potom si vymeňte úlohy.

Čo si sa naučil

- Chyby sú červené v okne **Output** a hovoria ti **číslo riadku**.
- `nil` znamená, že Roblox objekt nenašiel – skoro vždy je to preklep v mene.
- `print()` je tvoj najlepší nástroj na ladenie.

9. Lekcia – Premenné

Čo je premenná

Premenné používame, ak chceme, aby si program pamätal nejakú hodnotu. Premennú si môžeš predstaviť ako **krabičku v pamäti počítača**, do ktorej hodnotu vložíme. Neskôr sa do nej môžeme pozrieť alebo obsah vymeniť.

Každá premenná má **meno** a **hodnotu**. Vytvoríme ju takto:

```
local penize = 100
```

Vypísanie premennej do Outputu:

```
print(penize)
```

Zmena hodnoty premennej (slovo `local` sa už nepíše, premenná už existuje):

```
penize = 35
```

Čo znamená local

Slovo **`local`** píšeme **iba pri vytváraní** premennej. Hovorí: „táto premenná patrí len tomuto skriptu“.

Bez `local` by premenná bola spoločná pre celú hru a ľahko by sa pobila s inou premennou rovnakého mena. Navyše je taká premenná pomalšia.

Pravidlo: každú novú premennú vytváraj so `local`. Neskôr ti to ušetrí veľa hľadania chýb.

Typy premenných

Okrem čísel môžeme do premenných ukladať texty, logické hodnoty, vektory, farby aj celé objekty.

```
local cislo = 10
local text = "ahoj"
local pravda = true
local vektor = Vector3.new(0, 1, 0)
local objekt = workspace.Kvader
local poloha = workspace.Kvader.Position
```

Rovné úvodzovky

Text musí byť v **rovných úvodzovkách** `"ahoj"`.

Ak text napíšeš alebo skopíruješ s „oblými“ úvodzovkami – `„ahoj“` – Roblox to nepochopí a skript spadne. Toto sa stáva najmä pri kopírovaní z Wordu.

Pomenovanie premenných

- Meno má hovoriť, čo v premennej je: `penize` je lepšie ako `p`.
- V mene **nesmie byť medzera** – píš `pocetBodov`, nie `pocet bodov`.
- V mene **nesmie byť diakritika** – píš `penize`, nie `peniaze` s dĺžňom, a nikdy nie `počet`.
- Meno nesmie začínať číslom.

Matematické operácie

```
print(1 + 2)    -- vypíše 3
print(6 - 4)    -- vypíše 2
print(2 * 3)    -- vypíše 6, hviezdičkou označujeme násobenie
print(20 / 2)   -- vypíše 10
```

Zvýšenie hodnoty premennej

Oba tieto zápisy zvýšia hodnotu premennej o 1. Druhý je len kratší zápis toho istého.

```
penize = penize + 1
penize += 1
```

Rovnako vieme zvýšiť aj hodnotu vlastnosti objektu:

```
workspace.Kvader.Transparency = workspace.Kvader.Transparency + 0.1
workspace.Kvader.Transparency += 0.1
```

Spájanie textu

Ak chceš do jednej vety spojiť text a hodnotu premennej, použi **dve bodky** `..`

```
local penize = 100
print("V peňaženke mám " .. penize .. " eur.")
-- vypíše: V peňaženke mám 100 eur.
```

9.1 Úloha – Bydlisko a vek

Vytvor premennú pre bydlisko a ulož do nej svoje bydlisko. Vytvor druhú premennú pre vek a ulož do nej číslo.

Dbaj na správne a intuitívne pomenovanie premenných a nezabudni na `local`.

Vypíš tieto dve informácie na výstup použitím **2× `print()`**.

9.2 Úloha – Obľúbený šport a jedlo

Vytvor premennú, kde uložíš názov obľúbeného športu, a druhú na obľúbené jedlo.

Vypíš tieto dve informácie pomocou **1× `print()`** a spoj ich do jednej vety cez `..`

9.3 Úloha – Peňaženka

Nastav hodnotu peňazí na 100 a vypíš aktuálny stav peňaženky pred nákupom.

Po 2 sekundách sme fiktívne zrealizovali platbu – nastav hodnotu peňazí na 35 a vypíš stav po nákupe.

9.4 Úloha – Neposlušná guľa

Guľu aj informačnú správu maj od začiatku uložené v samostatných premenných.

Po 3 sekundách zmeň farbu gule. Po ďalších 3 sekundách zapni ukotvenie a vypíš o tom informačnú správu. Po 3 sekundách ukotvenie vypni.

9.5 Úloha – Aký je dnešný dátum?

Ulož do premennej dnešný dátum a vypíš ho na výstup aj s vetou.

```
local dnes = os.date("%d.%m.%Y")
print("Dnes je " .. dnes)
```

9.6 Bonus – Počítadlo

Vytvor premennú `pocet` s hodnotou 0. Potom ju päťkrát za sebou zvýš o 1 pomocou `+=` a po každom zvýšení ju vypíš. Medzi výpismi počkaj pol sekundy (`task.wait(0.5)`).

Čo si sa naučil

- Premenná je pomenovaná krabička na hodnotu.
- Novú premennú vytváraj vždy so slovom **`local`**.
- Mená bez medzier a bez diakritiky.
- Texty spájame dvoma bodkami `..`

10. Lekcia – Funkcie

Vysvetlenie funkcií

Funkcia je pomenovaný kúsok programu, ktorý vieme kedykoľvek spustiť. Napíšeme ju raz a použiť ju môžeme koľkokrát chceme.

```
local function nazovFunkcie()
    print("toto je funkcia")
end
```

Funkcia začína **hlavičkou** (1. riadok) a končí kľúčovým slovom **`end`** (3. riadok). Riadky medzi hlavičkou a `end` sú **telom** funkcie – môže ísť o jeden alebo viac príkazov.

Keď spustíme tento kód, nič sa nevypíše – napriek tomu, že tam je `print`. Je to preto, že príkazy v tele funkcie sa nevykonajú. Len sa do pamäte uloží, že taká funkcia existuje.

Na to, aby sa príkazy v tele funkcie vykonali, musíme funkciu **zavolať**. Voláme ju tak, že napíšeme jej názov a okrúhle zátvorky:

```
nazovFunkcie()
```

Funkciu treba najprv vytvoriť, až potom zavolať

Volanie funkcie musí byť v skripte **nižšie** než jej vytvorenie. Ak funkciu zavoláš skôr, než ju vytvoríš, Roblox ju nenájde.

Vstupný argument vo funkcii

Často potrebujeme dostať do funkcie nejakú hodnotu. Vtedy použijeme **vstupný argument**:

```
local function pozdrav(text)
    print(text)
end

pozdrav("Ahoj")     -- vypíše Ahoj
pozdrav("Dobrý deň")  -- vypíše Dobrý deň
```

Zavoláme funkciu a do nej ide priamo text. Ten máme vďaka vstupnému argumentu vnútri funkcie k dispozícii a vieme s ním pracovať. Všimni si, že tú istú funkciu sme použili dvakrát s inou hodnotou.

Návratová (return) hodnota funkcie

Funkcie nám umožňujú aj **vrátiť hodnotu** naspäť. S ňou potom vieme v kóde ďalej pracovať. Ukážeme si príklad, kde k vstupnému číslu pripočítame 10 a túto hodnotu vrátime:

```
local function pripocitajDesat(cislo)
    return cislo + 10
end

local vysledok = pripocitajDesat(25)
print(vysledok)   -- vypíše 35
```

Vrátenú hodnotu sme si uložili do premennej `vysledok` a následne ju vypísali na výstup.

Kedy použiť funkciu

Vždy, keď ten istý kód píšeš **druhýkrát**. Namiesto kopírovania z neho sprav funkciu.

Keby si potom potreboval niečo zmeniť, opravíš to na jednom mieste, nie na piatich.

10.1 Úloha – Vypíš meno objektu

Vytvor funkciu, v ktorej sa vypíše meno rodičovského objektu (`script.Parent.Name`). Túto funkciu v skripte následne zavolaj.

10.2 Úloha – Guľa stoj!

Po úvodnom výpise počkaj 3 sekundy a následne zavolaj funkciu na zastavenie gule.

V samotnej funkcii nastav správnu vlastnosť a hodnotu pre ukotvenie gule a pridaj aj informačný výpis.

10.3 Úloha – Posielame novú farbu

Vytvor funkciu, ktorá bude mať **tri vstupné argumenty** – zložky farby (červená, zelená, modrá).

Pomocou týchto argumentov vo funkcii zmeň farbu objektu. Zavolaj funkciu s tebou zvolenými hodnotami.

Nápoveda: `Color3.fromRGB(cervena, zelena, modra)`

10.4 Úloha – Sčítavacia kalkulačka

Vytvor funkciu s dvoma vstupnými argumentmi – číslami. V tele funkcie vykonaj súčet a ten bude zároveň návratová hodnota.

Zavolaj funkciu so vstupnými číslami, návratovú hodnotu ulož do premennej a výsledok vypíš.

10.5 Bonus – Kalkulačka na štyri operácie

Vytvor štyri funkcie – `scitaj`, `odcitaj`, `vynasob`, `vydel`. Každá má dva vstupné argumenty a vracia výsledok. Otestuj všetky štyri.

Čo si sa naučil

- Funkcia sa vytvára cez `local function meno() ... end`.
- Sama sa nevykoná – treba ju **zavolať**.
- **Vstupný argument** posiela hodnotu dovnútra, **`return`** vracia hodnotu von.

11. Lekcia – Podmienky

Vetvenie programu pomocou podmienok

Podmienky slúžia na **vetvenie** programu. Zisťujeme, či niečo platí alebo neplatí, a podľa toho sa program zachová.

```
if <podmienka> then
    -- toto sa vykoná, ak podmienka platí
else
    -- toto sa vykoná, ak podmienka neplatí
end
```

Kľúčové sú slová **`if`** a **`else`**, sprevádzané slovom **`then`** a koncovým **`end`**. Bez nich to nebude fungovať.

Porovnávacie operátory

Podmienka je otázka, na ktorú sa dá odpovedať áno alebo nie. Skladáme ju z týchto operátorov:

Operátor	Význam	Príklad
`==`	rovná sa	`if vek == 12 then`
`~=`	nerovná sa	`if meno ~= "Peter" then`
`<`	je menšie ako	`if penize < 50 then`
`>`	je väčšie ako	`if body > 100 then`
`<=`	je menšie alebo rovné	`if vek <= 15 then`
`>=`	je väčšie alebo rovné	`if penize >= cena then`

Jedno rovnítko verzus dve

`=` znamená *„priraď“* – `penize = 100` vloží do premennej stovku.

`==` znamená *„rovná sa?“* – `if penize == 100 then` sa pýta, či je v premennej stovka.

Zamieňanie týchto dvoch je najčastejšia chyba pri podmienkach. Ak ti Roblox pri `if` hlási chybu, skontroluj najprv toto.

Logické operátory

Podmienky sa dajú spájať:

Operátor	Význam	Kedy platí
`and`	a zároveň	Musia platiť **obe** podmienky
`or`	alebo	Stačí, ak platí **aspoň jedna**
`not`	nie	Obráti platnosť na opačnú

```
if penize >= cena and vek >= 12 then
    print("Môžeš si to kúpiť.")
end
```

Viac možností: elseif

Ak potrebuješ viac než dve možnosti, použi **`elseif`**. Program ich skúša zhora nadol a zastaví sa pri prvej, ktorá platí.

```
local body = 75

if body >= 90 then
    print("Výborne!")
elseif body >= 70 then
    print("Dobre.")
elseif body >= 50 then
    print("Ešte to ide.")
else
    print("Skús to znova.")
end
```

`elseif` môže byť koľko chceš, `else` je najviac jeden a je vždy posledný. Celá stavba sa uzatvára jediným `end`.

11.1 Úloha – Kúpime si šmykľavku?

Do premenných ulož cenu šmykľavky a peniaze na účte.

Pomocou podmienky zisti, či si ju môžeme kúpiť. V oboch prípadoch vypíš informačnú správu.

Ak máme dostatok peňazí, šmykľavku zafarbi na zeleno, v opačnom prípade na červeno.

11.2 Úloha – Môžeš vstúpiť?

Do premennej ulož vek. Ak je vek 15 alebo viac, vypíš `Môžeš vstúpiť`, inak vypíš `Ešte nie`.

Vyskúšaj si program s troma rôznymi hodnotami veku.

11.3 Úloha – Známkovanie

Do premennej ulož počet bodov (0 až 100). Pomocou `elseif` vypíš známku:

- 90 a viac → `1`
- 75 až 89 → `2`
- 60 až 74 → `3`
- 45 až 59 → `4`
- menej ako 45 → `5`

11.4 Úloha – Dve podmienky naraz

Vytvor premenné `maKluc` (hodnota `true` alebo `false`) a `penize`.

Dvere sa otvoria len vtedy, ak má hráč kľúč **a zároveň** má aspoň 50 peňazí. Použi `and` a vypíš, čo sa stalo.

11.5 Úloha – Semafór

Do premennej ulož číslo 1, 2 alebo 3. Pomocou `elseif` zafarbi objekt: 1 → zelená, 2 → oranžová, 3 → červená.

Ak je v premennej iné číslo, vypíš `Neznáma farba`.

11.6 Bonus – Kontrola v funkcii

Vytvor funkciu `mozeKupit(penize, cena)`, ktorá **vráti** `true` alebo `false`. Potom ju použi v podmienke:

```
if mozeKupit(100, 80) then
print("Kupujeme!")
end
```

Čo si sa naučil

- `if / then / else / end` vetví program.
- `=` priraďuje, **`==` porovnáva**.
- `and`, `or`, `not` spájajú podmienky.
- `elseif` rieši viac než dve možnosti.

12. Lekcia – Mini-projekt: Prekážková dráha

Čo dnes robíme

Dnes nepreberáme nič nové. Dnes použiješ všetko, čo vieš z lekcií 1 až 11, a postavíš z toho jednu vec. Toto je prvá príležitosť postaviť niečo vlastné. Využi ju.

Zadanie

Postav **prekážkovú dráhu**, ktorou musí hráč prejsť od štartu do cieľa. Dráha musí obsahovať aspoň päť prekážok a musí sa dať prejsť – ak sa to nedá, nie je to dráha, ale hlavolam.

Povinné prvky

Prvok	Koľko
Objekty tvarované nástrojmi Move / Scale / Rotate	aspoň 8
Premenné vytvorené so `local`	aspoň 3
Vlastná funkcia	aspoň 1
Podmienka `if`	aspoň 1
Skript, ktorý mení vlastnosti objektu počas hry	aspoň 1
Komentáre vysvetľujúce, čo kód robí	aspoň 3

Ako na to – postup po krokoch

Nakresli si dráhu na papier. Kde je štart, kde cieľ, čo je medzi tým. Trvá to päť minút a ušetrí ti pol hodiny.

Postav najprv trasu – plošiny, po ktorých sa dá prejsť. Otestuj, či sa dá prejsť. Až potom pridávaj ozdoby.

Pridaj jednu prekážku so skriptom. Otestuj ju. Až keď funguje, pridaj ďalšiu.

Nechaj to prejsť spolužiaka. Zistíš veci, ktoré by ti nenapadli.

Najdôležitejšia rada

Nestavaj celú dráhu a potom ju testuj. Keď v nej bude desať skriptov a niečo nebude fungovať, nezistíš, čo.

Postav jednu vec. Otestuj. Funguje? Postav ďalšiu.

Takto vieš po každom kroku, ktorá zmena to pokazila – tá posledná.

Nápady na prekážky

- Miznúca plošina – po pár sekundách sa jej nastaví `Transparency` a `CanCollide`
- Meniaca sa farba – objekt strieda farby, aby bolo vidieť, kedy je „bezpečný“
- Pohyblivá plošina – mení sa jej `Position`
- Falošná cesta – vyzerá pevne, ale má `CanCollide = false`

- Tabuľa s nápisom – objekt, ktorý po chvíli vypíše do konzoly nápovedu

12.1 Úloha – Návrh

Nakresli dráhu na papier a ukáž ju učiteľovi, než začneš stavať.

12.2 Úloha – Trasa

Postav trasu z aspoň 8 objektov a over v hre, že sa dá prejsť od štartu do cieľa.

12.3 Úloha – Oživenie

Doplň skripty tak, aby dráha spĺňala všetky povinné prvky z tabuľky vyššie.

12.4 Úloha – Testovanie

Nechaj dráhu prejsť spolužiaka. Zapíš si, kde sa zasekol, a aspoň jednu vec oprav.

12.5 Bonus – Časomiera

Doplň do dráhy počítadlo pokusov: premennú, ktorá sa zvýši vždy, keď hráč začne odznova, a vypíše sa do konzoly.

13. Lekcia – ScreenGui a MouseButton1Click

ScreenGui

ScreenGui nám umožňuje pridať priamo na obrazovku text, tlačidlo alebo obrázok. Pridáme ho tak, že pôjdeme kurzorom myši na priečinok **StarterGui** v Exploreri a klikneme na plusko „+“. Z ponuky vyberieme **ScreenGui**.

Do ScreenGui môžeme rovnakým spôsobom pridať:

Prvok	Čo to je
`TextLabel`	text
`ImageLabel`	obrázok
`TextButton`	tlačidlo
`Frame`	obdĺžnik, do ktorého môžeme pridať texty, obrázky a tlačidlá

Vzhľad týchto prvkov meníme pomocou vlastností:

Vlastnosť	Čo robí
`Text`	text, ktorý sa zobrazí v danom prvku
`TextSize`	veľkosť textu
`TextColor3`	farba textu
`Font`	typ písma
`BackgroundColor3`	farba pozadia
`BackgroundTransparency`	priehľadnosť pozadia
`Visible`	viditeľnosť prvku

LocalScript – a prečo nie obyčajný Script

Pod prvok GUI budeme vkladať **LocalScript**, nie obyčajný Script. Vzhľad a správanie prvkov budeme meniť takto:

```
script.Parent.Text = "Náš text"
```

Prečo LocalScript?

Obrazovku vidí **len jeden konkrétny hráč** – je to jeho obrazovka, nie spoločná časť sveta.

Preto kód, ktorý ju mení, musí bežať **v počítači toho hráča**. Na to slúži **LocalScript**.

Obyčajný **Script** beží na **serveri** a je spoločný pre všetkých hráčov. Preto ho používame na objekty vo svete (steny, plošiny) a LocalScript na obrazovku a ovládanie.

Toto je jeden z najdôležitejších rozdielov v Robloxe. Podrobne sa mu budeme venovať v pokračovaní kurzu.

Vlastnosť MouseButton1Click

`TextButton` má udalosť **`MouseButton1Click`**, pomocou ktorej vieme nastaviť, aby sa niečo stalo, keď hráč stlačí tlačidlo.

```
script.Parent.MouseButton1Click:Connect(function()
    -- toto sa stane, keď stlačíme tlačidlo
end)
```

Zápis `:Connect(...)` znamená *„pripoj k tejto udalosti túto funkciu“*. Funkcia sa nespustí hneď – čaká, kým hráč klikne.

13.1 Úloha – Schováme text

Po 3 sekundách od spustenia hry schovaj `TextLabel` nastavením vlastnosti `Visible` na správnu hodnotu.

13.2 Úloha – Privítanie v hre

Po spustení hry bude hráč vidieť text s privítacou správou.

Po 3 sekundách zmeň text správy a zmeň farbu textu. Po ďalších 2 sekundách správu schovaj.

13.3 Úloha – Čarovné tlačidlo na zmiznutie

Naprogramuj tlačidlo tak, aby sa po kliknutí naň nastavila `Transparency` objektu na maximálnu hodnotu.

13.4 Bonus – Prepínač

Naprogramuj jedno tlačidlo tak, aby objekt striedavo skrývalo a zase ukazovalo. Budeš potrebovať premennú, ktorá si pamätá, v akom stave objekt práve je, a podmienku.

Čo si sa naučil

- `ScreenGui` sa pridáva do `StarterGui`.
- Obrazovku mení **LocalScript**, svet mení **Script**.
- `MouseButton1Click:Connect(...)` reaguje na kliknutie.

14. Lekcia – Udalosť Touched

Vysvetlenie udalosti

Udalosť **Touched** nastane vtedy, keď sa niečo dotkne objektu. Ak ju chceme použiť, musíme najprv určiť, ktorý objekt bude na dotyk reagovať. Napíšeme cestu k tomuto objektu (relatívnu alebo absolútnu, rovnako ako pri nastavovaní vlastností) a za ňou `.Touched:Connect`.

```
script.Parent.Touched:Connect(function(hit)
    -- príkazy, ktoré sa vykonajú pri dotyku
end)
```

Čo je hit

Slovo **`hit`** v zátvorke je **vstupný argument udalosti**. Udalosť nám ním povie, **čo presne sa objektu dotklo**. Vďaka nemu vieme zistiť, či to bola napríklad ľavá alebo pravá noha hráča:

```
script.Parent.Touched:Connect(function(hit)
    if hit.Name == "LeftFoot" then     -- alebo RightFoot
        -- príkazy, ktoré sa vykonajú pri dotyku ľavou nohou
    end
end)
```

Je to ten istý princíp ako vstupný argument funkcie z 10. lekcie – len hodnotu do funkcie neposielaš ty, ale samotná hra.

Touched sa spúšťa veľmi často

Kým hráč stojí na plošine, udalosť Touched sa môže spustiť **mnohokrát za sekundu**.

Ak teda v nej pridávaš body, hráčovi ich pribudnú stovky. Preto je dobré si pomocou premennej pamätať, či sa to už raz stalo.

14.1 Úloha – Platformy na zmenu tvaru objektu

Pridaj do hry dve platformy (vytvaruj ich) a ešte jeden objekt, napríklad kváder.

Naprogramuj platformy tak, že ak sa hráč dotkne **ľavou nohou** prvej platformy, tvar objektu sa zmení na valec.

Ak sa dotkne **pravou nohou** druhej platformy, tvar objektu sa zmení na guľu.

14.2 Úloha – Pridávame a uberáme zdravie

Vytvor dve platformy. Prvej nastav zelenú farbu – tá bude pridávať životy.

Ak sa hráč dotkne prvej platformy pravou nohou, pridá sa mu 10 bodov života.

Ak sa dotkne druhej platformy ľavou nohou, zmení sa jej farba na červenú a hráčovi sa uberie 10 bodov života.

14.3 Bonus – Teleport

Vytvor platformu, ktorá po dotyku presunie hráča na iné miesto v mape (zmenou jeho `Position`). Dávaj pozor, aby ho neteleportovala donekonečna dokola.

Čo si sa naučil

- `Touched:Connect(function(hit) ... end)` reaguje na dotyk.
- `hit` hovorí, **čo** sa objektu dotklo.
- Udalosť sa môže spustiť mnohokrát za sekundu.

15. Lekcia – Spracovanie vstupu od hráča

Vysvetlenie spracovania vstupu

Doteraz hra reagovala na dotyk alebo na kliknutie na tlačidlo. Teraz sa naučíme reagovať priamo na **klávesnicu a myš**.

Do **StarterPlayer → StarterPlayerScripts** vlož **LocalScript** s týmto obsahom:

```
local UIS = game:GetService("UserInputService")

UIS.InputBegan:Connect(function(input)
    if input.KeyCode == Enum.KeyCode.Q then
        -- príkazy, ktoré sa vykonajú, ak hráč stlačil Q
    end

    if input.KeyCode == Enum.KeyCode.Space then
        -- príkazy, ktoré sa vykonajú, ak hráč stlačil medzeru
    end

    if input.UserInputType == Enum.UserInputType.MouseButton1 then
        -- príkazy, ktoré sa vykonajú, ak hráč stlačil ľavé tlačidlo myši
    end
end)
```

Aj tu máme vstupný argument – volá sa **`input`** a nesie informáciu o tom, čo hráč stlačil.

> **Prečo si službu ukladáme do premennej**
>
> Zápis `game:GetService("UserInputService")` je dlhý. Preto si ho hneď na začiatku uložíme do premennej `UIS` a ďalej píšeme len tie tri písmená.
>
> Je to presne to, na čo sú premenné – aby sme dlhé veci nemuseli písať stále dokola.

Zistenie, na ktorý objekt hráč klikol

V prípade stlačenia tlačidla myši môžeme zistiť, na ktorý objekt hráč klikol:

```
UIS.InputBegan:Connect(function(input)
    if input.UserInputType == Enum.UserInputType.MouseButton1 then
        local ciel = game.Players.LocalPlayer:GetMouse().Target
        -- v premennej ciel je objekt, na ktorý hráč klikol
    end
end)
```

Klik do prázdna

Ak hráč klikne do neba alebo do prázdna, `Target` bude **`nil`** – teda nič.

Ak potom skúsiš nastaviť `ciel.Material`, skript spadne s chybou `attempt to index nil`. Poznáš ju z 8. lekcie.

Preto sa vždy najprv opýtaj, či tam vôbec niečo je:

```
if ciel then
ciel.Material = Enum.Material.Brick
end
```

15.1 Úloha – Zázračné tlačidlá na klávesnici

Pridaj do hry guľu. Ak hráč stlačí na klávesnici `K`, guľa zmizne a nebudeme sa jej vedieť dotknúť (správne nastavenie `Transparency` a `CanCollide`).

Ak stlačí `L`, guľa bude opäť viditeľná a bude sa dať dotknúť.

15.2 Úloha – Meníme materiál pomocou myšky

Pridaj do hry kocku alebo ľubovoľný objekt. Ak hráč klikne ľavým tlačidlom myši, zmení sa materiál objektu na tehlu (`Brick`).

Nezabudni ošetriť klik do prázdna.

15.3 Úloha – Lávové objekty

Pridaj do hry objekty (alebo použi tie z predošlých úloh). Ak hráč stlačí **stredné tlačidlo myši** a ukazuje na objekt, materiál tohto objektu sa zmení na lávu (`CrackedLava`).

Nápoveda: `Enum.UserInputType.MouseButton3`

15.4 Bonus – Farebná klávesnica

Naprogramuj klávesy `1`, `2` a `3` tak, aby každá zafarbila objekt inou farbou. Použi `Enum.KeyCode.One`, `Two`, `Three`.

Čo si sa naučil

- `UserInputService` a udalosť `InputBegan` reagujú na klávesnicu a myš.
- `input.KeyCode` hovorí, ktorá klávesa; `input.UserInputType` hovorí, ktoré tlačidlo myši.
- Pred prácou s `Target` vždy over, či nie je `nil`.

16. Lekcia – For cyklus

Použitie for cyklu

Cykly používame na opakovanie nejakej postupnosti príkazov. **For cyklus** je špecifický tým, že dopredu zadefinujeme, koľko opakovaní (iterácií) sa vykoná.

```
for i = 1, 5 do
    -- táto postupnosť príkazov sa 5-krát zopakuje
end
```

Premenná `i` bude nadobúdať hodnoty 1 až 5. Po každom prejdení cyklu sa jej hodnota zvýši o 1. Tejto premennej hovoríme **riadiaca premenná**.

```
for i = 1, 5 do
    print(i)
end
-- vypíše postupne 1, 2, 3, 4, 5
```

Cyklus s čakaním

Ak chceš, aby sa medzi opakovaniami niečo dialo viditeľne, pridaj do cyklu čakanie:

```
for i = 1, 5 do
print(i)
task.wait(1)
end
```

For cyklus pre časti modelu

Pomocou for cyklu môžeme prejsť všetky časti modelu. Funkciou `:IsA("BasePart")` testujeme, či je daná časť skutočný objekt – aby sme neprechádzali napríklad aj skripty.

```
for _, item in ipairs(script.Parent:GetDescendants()) do
    if item:IsA("BasePart") then
        -- čo sa má stať s každou časťou modelu
        -- jednotlivé časti modelu sú uložené v premennej item
    end
end
```

Podčiarkovník _ na začiatku je premenná, ktorú **nepotrebujeme**. Je to zvyk programátorov: keď nejakú hodnotu dostávame, ale nezaujíma nás, pomenujeme ju _, aby bolo hneď vidieť, že s ňou nepracujeme.

16.1 Úloha – Zapnutie časomiery hry

Na `TextLabel` pomocou for cyklu aktualizuj čas každú sekundu až do 15 sekúnd.

Potom nech je na `TextLabel` správa `Vypiname casomieru`.

16.2 Úloha – Odpočítavanie času pre zmiznutie objektu

Pridaj do hry objekt. Na `TextLabel` sprav pomocou for cyklu odpočítavanie sekúnd od 10.

Počas každej iterácie zároveň znižuj viditeľnosť objektu.

Keď čas vyprší (cyklus skončí), `TextLabel` bude ukazovať `Cas vyprsal, objekt je neviditelny`.

16.3 Úloha – Bláznivé farby semafóra

Pridaj do hry objekt. Pomocou 10 iterácií vo for cykle simuluj semafor – každú sekundu sa bude meniť farba objektu (zelená, oranžová, červená).

16.4 Úloha – Objekty modelu, nech ste preč!

Pridaj do hry **Model** a doň viacero objektov. V skripte v Modeli najskôr počkaj 3 sekundy.

Potom pomocou for cyklu prejdi všetky objekty v Modeli a nastav im `Transparency` na hodnotu 1. Objekty budú postupne miznúť.

16.5 Bonus – Schodisko

Pomocou for cyklu vytvor schodisko – v cykle vytvor 10 nových objektov (`Instance.new("Part")`), z ktorých každý bude o kúsok vyššie a ďalej ako predošlý.

Nápoveda: pozíciu počítaj z riadiacej premennej `i`.

Čo si sa naučil

- `for i = 1, 5 do ... end` zopakuje kód päťkrát.
- **Riadiaca premenná** `i` hovorí, v ktorom opakovaní práve sme.
- `ipairs` + `GetDescendants()` prejde všetky časti modelu.

17. Lekcia – While cyklus

Použitie while cyklu

Pri **while cykle** sa postupnosť opakuje, **kým platí podmienka**. Na rozdiel od for cyklu nemusí byť dopredu zrejmé, koľko opakovaní počas behu programu nastane.

```
while <podmienka> do
    -- táto postupnosť sa opakuje, kým platí podmienka
end
```

Príklad:

```
local pocet = 0

while pocet < 5 do
    print(pocet)
    pocet += 1
    task.wait(1)
end
```

Nekonečný cyklus

Pomocou while cyklu môžeme jednoducho spraviť **nekonečný cyklus**, ktorý beží počas celého trvania hry:

```
while true do
    -- postupnosť príkazov, ktorá sa bude opakovať stále
    task.wait(1)
end
```

Bez task.wait() hra zamrzne!

Toto je najnebezpečnejšia chyba v celej učebnici. Ak napíšeš nekonečný cyklus **bez čakania**:

```
while true do
print("ahoj")
end
```

...cyklus zaberie celý počítač, hra prestane reagovať a Roblox Studio budeš musieť vypnúť násilím. Rozrobenú prácu stratíš.

V každom nekonečnom cykle musí byť `task.wait()`. Bez výnimky.

For alebo while?

Použi **`for`**, keď dopredu vieš, koľkokrát sa má niečo zopakovať (10-krát, pre každý objekt modelu).

Použi **`while`**, keď to dopredu nevieš a opakuješ, kým niečo platí (kým má hráč životy, kým hra beží).

17.1 Úloha – Počítame až po limit

Do `TextLabel` naprogramuj počítadlo stlačení klávesy `Q` a každé stlačenie vypíš do konzoly.

V tom istom LocalScripte použi while cyklus, v ktorom budeš počítať sekundy a tie budú ako text v `TextLabel`.

Podmienka pre while cyklus je, že počítame, kým je počet stlačení `Q` menší ako 20.

17.2 Úloha – Nekonečné zmeny objektu

Na začiatku skriptu počkaj 3 sekundy. Potom naprogramuj nekonečný while cyklus, v ktorom budeš meniť vlastnosti objektu. Po každej zmene pridaj `task.wait(1)`.

V cykle naprogramuj tieto zmeny: zmeň farbu na zelenú, potom materiál na kameň (`Rock`), tvar na guľu (`Ball`), farbu na červenú, materiál na asfalt (`Asphalt`) a nakoniec tvar na valec (`Cylinder`).

17.3 Bonus – Blikajúce svetlo

Naprogramuj objekt, ktorý neustále bliká – strieda materiál Neon a obyčajný materiál. Rýchlosť blikania si nastav premennou, aby sa dala ľahko zmeniť na jednom mieste.

Čo si sa naučil

- `while <podmienka> do ... end` opakuje, kým podmienka platí.
- `while true do` je nekonečný cyklus.
- **V nekonečnom cykle musí byť `task.wait()`**, inak hra zamrzne.

18. Lekcia – Záverečný projekt

Toto je tvoja hra

Posledné dve stretnutia patria tebe. Postavíš vlastnú hru a dáš ju zahrať spolužiakom.

Zadanie

Postav **vlastnú mini-hru**. Môže to byť prekážková dráha, hľadanie pokladu, úniková miestnosť, pretek s časom alebo čokoľvek iné – dôležité je, aby mala **cieľ** (hráč vie, čo má dosiahnuť) a aby sa dala **dohrať**.

Úrovne – vyber si, na čo si trúfaš

Úroveň	Čo musí hra mať
Bronz	Aspoň 10 objektov · 3 premenné so `local` · 1 podmienka · 1 udalosť · hra sa dá dohrať
Striebro	Bronz + vlastná funkcia · 1 cyklus · prvok na obrazovke (`TextLabel`)
Zlato	Striebro + reakcia na klávesnicu alebo myš · počítadlo bodov · komentáre v kóde

Postup

- **Nápad a návrh.** Jednou vetou napíš, čo má hráč v tvojej hre spraviť. Ak sa ti tá veta píše ťažko, nápad je ešte hmlistý – zúž ho.
- **Stavba sveta.** Najprv trasa a hlavné objekty, potom detaily.
- **Skripty po jednom.** Napíš jeden, otestuj, až potom ďalší.
- **Testovanie u spolužiaka.** Ty svoju hru poznáš a preto ju nevieš pokaziť. Cudzí človek nájde chybu za desať sekúnd.
- **Doladenie.** Oprav aspoň jednu vec, ktorú ti spolužiak našiel.

Zdieľanie hry so spolužiakmi

Aby si hru mohol dať zahrať ostatným, musíš ju nahrať na Roblox:

- V Roblox Studiu klikni na **File → Publish to Roblox**
- Vyplň formulár a stlač **Create**
- Choď na **www.roblox.com** a prihlás sa
- Klikni na **Create**, potom na názov alebo obrázok svojej hry
- V **Content Settings** nastav hru na **Public** a klikni **Save**
- Vráť sa do sekcie **Creations**, klikni na „…“ pri obrázku hry a zvoľ **Copy URL**
- Odkaz pošli spolužiakom – môžu si tvoju hru zahrať

Než hru zverejníš

Do názvu ani popisu hry **nedávaj svoje priezvisko, adresu ani školu**. Stačí prezývka.

Zverejnená hra je verejná – uvidí ju ktokoľvek na internete, nielen spolužiaci.

Otázky pri testovaní hry spolužiaka

- Pochopil si bez vysvetľovania, čo máš v hre robiť?
- Čo sa ti na hre páčilo a čo nie?
- Podarilo sa ti hru pokaziť alebo sa niekde zaseknúť? Ako?
- Máš nápad, ako hru zlepšiť?

Prezentácia (2 minúty)

- Ako sa hra volá a čo v nej má hráč spraviť.
- Ukáž ju v akcii.
- Čo bolo **najťažšie** a ako si to vyriešil.
- Čo by si pridal, keby si mal ešte dve hodiny.

18.1 Úloha – Návrh hry

Napíš jednou vetou, čo má hráč v tvojej hre spraviť, a nakresli návrh na papier. Ukáž ho učiteľovi.

18.2 Úloha – Stavba

Postav hru aspoň na úroveň **Bronz**. Zameraj sa na to, aby fungovala – nie na to, aby vedela všetko.

18.3 Úloha – Zverejnenie

Zverejni svoju hru a pošli odkaz do skupiny.

18.4 Úloha – Testujeme hry

Otestuj hry aspoň dvoch spolužiakov a odpovedz im na štyri otázky vyššie. Aspoň jednu vec, ktorú ti povedia, oprav.

18.5 Úloha – Prezentácia

Predstav svoju hru skupine.

Čo si dokázal

- Vieš stavať a tvarovať herný svet.
- Vieš písať skripty v jazyku Lua: premenné, podmienky, funkcie, cykly, udalosti.
- Vieš prečítať chybovú hlášku a nájsť, kde je problém.
- Postavil si vlastnú hru, ktorú si niekto zahral. To dokáže málokto.

Časť B – Bonusové lekcie

Čo je Časť B

Tieto lekcie **nie sú súčasťou 16 stretnutí krúžku**. Nezmestili sa do času, ale sú príliš dobré na to, aby chýbali.

Môžeš si ich spraviť **doma** – Roblox Studio je zadarmo a všetko potrebné už vieš.

Preberieme ich v **pokračovaní kurzu**, ak sa doň prihlásiš.

Ak niektorú zvládneš sám, pokojne ju použi aj v záverečnom projekte. Zvuk (19. lekcia) je na to najjednoduchší.

19. Lekcia – Prehrávanie zvuku

Najjednoduchšia z bonusových lekcií. Zvládneš ju za pol hodiny a tvoja hra bude hneď o triedu lepšia.

Pridanie zvuku

Otvor **Toolbox**, vyhľadaj zvuk (**Audio**) a kliknutím naň ho pridaj do **Workspace**.

Ak chceš, aby sa zvuk prehral hneď po spustení hry, stačí zakliknúť vlastnosť **`Playing`**, prípadne aj **`Looped`**, ak sa má po prehratí spustiť znova od začiatku.

> **Nie každý zvuk sa dá použiť**
>
> Roblox v posledných rokoch výrazne obmedzil používanie zvukov. Väčšina nahraných zvukov je **súkromná** a v tvojej hre sa neprehrá – aj keď ju v Toolboxe vidíš.
>
> Ak sa zvuk neprehrá a v Outpute nie je chyba, skoro určite je to týmto. Skús iný zvuk.
>
> Učiteľ ti dá zoznam overených zvukov, ktoré fungujú.

Prehrávanie zo skriptu

Ak chceš zvuk prehrať až vtedy, keď nastane nejaká udalosť, potrebuješ skript. Funkciou `:Play()` zvuk prehráš, funkciou `:Stop()` zastavíš.

```
workspace.MenoZvuku:Play()             -- prehratie zvuku
workspace.MenoZvuku:Stop()             -- zastavenie zvuku
workspace.MenoZvuku.PlaybackSpeed = 3  -- 3-krát väčšia rýchlosť prehrávania
workspace.MenoZvuku.Volume = 0.5       -- polovičná hlasitosť
```

> **19.1 Úloha – Hrá to, hrá to**
>
> Pridaj do hry zvuk. V skripte najskôr počkaj 3 sekundy a potom spusti zvuk. Následne po 5 sekundách zvuk zastav.

> **19.2 Úloha – Pusti to rýchlejšie a slabšie**
>
> Pridaj do hry iný zvuk. V skripte počkaj 3 sekundy, potom ulož zvuk do premennej a spusti ho.
>
> Po 3 sekundách zrýchli prehrávanie 2-krát. Po ďalších 3 sekundách zníž hlasitosť na hodnotu 0.2. Po ďalších 3 sekundách zvuk zastav.

> **19.3 Bonus – Zvuk pri dotyku**
>
> Spoj to s 14. lekciou: naprogramuj plošinu, ktorá pri dotyku prehrá zvuk. Použi udalosť `Touched`.

Čo si sa naučil

- Zvuk sa pridáva cez Toolbox do Workspace.
- `:Play()`, `:Stop()`, `PlaybackSpeed`, `Volume`.

20. Lekcia – Pridanie NPC postavy

NPC je postava v hre, ktorú neovláda hráč. V tejto lekcii jej dáme aj interaktívne okno s otázkou.

> **Toto je najťažšia lekcia v učebnici**
>
> Spája dokopy GUI, LocalScript, spracovanie vstupu aj vnorené podmienky. Ak sa v nej zasekneš, je to normálne.
>
> Odporúčame ju robiť až vtedy, keď máš 13. a 15. lekciu naozaj v malíčku.

Ako pridať do hry NPC

Klikni na sekciu **Avatar** a potom na tlačidlo **Rig Builder**. Zobrazí sa ponuka. Pre naše testovacie účely vyber:

- **Rig Type** – R15
- **Body Shape** – Masculine
- **Typ avataru** – Block Avatar

Pridanie interaktívneho okna

Cieľom je vytvoriť okno, ktoré sa zobrazí po kliknutí ľavým tlačidlom myši na NPC postavu.

V **StarterGui** klikni „+“ a pridaj **ScreenGui**
Pri ScreenGui klikni „+“ a pridaj **Frame** – bude slúžiť na zoskupenie textu a tlačidiel, aby sme ich vedeli naraz skryť a ukázať
Do Frame-u pridaj dva **TextButton**y – tlačidlá na klikanie
Neskôr pri programovaní do týchto TextButtonov pridáme samostatné **LocalScripty**

Vizuálne môže naše interaktívne okno vyzerať napríklad ako na obrázku. Vieme si tam spraviť ľubovoľné zmeny farieb, veľkosti okna aj textu.

> **Frame musí byť na začiatku skrytý**
>
> Než sa pustíš do programovania, nastav Frame-u vlastnosť **`Visible` na `false`** (nezaškrtnutý checkbox).
>
> Okno chceme vidieť až vtedy, keď naň klikneme – nie hneď po spustení hry.

Naprogramovanie funkcionality

V **StarterPlayer → StarterPlayerScripts** vytvor **LocalScript**:

```
local UIS = game:GetService("UserInputService")
local hrac = game.Players.LocalPlayer

UIS.InputBegan:Connect(function(input)
    if input.UserInputType == Enum.UserInputType.MouseButton1 then
        local ciel = hrac:GetMouse().Target

        if ciel and ciel.Parent.Name == "MenoNPC" then
            hrac.PlayerGui.ScreenGui.Frame.Visible = true
        end
    end
end)
```

V prvej podmienke zisťujeme, či je typ vstupu ľavé tlačidlo myši. V druhej overujeme, či sme klikli práve na NPC postavu – a ak áno, zobrazíme Frame.

Prečo tam je ciel and

Zápis `if ciel and ciel.Parent.Name == ...` najprv overí, či hráč vôbec na niečo klikol.

Ak by tam `ciel and` nebolo a hráč by klikol do neba, skript by spadol s chybou `attempt to index nil`. Poznáš ju z 8. lekcie.

Aby sme dosiahli kompletné správanie, treba naprogramovať aj logiku pri stlačení tlačidla. Na to použijeme `MouseButton1Click` z 13. lekcie. Nesmieme zabudnúť, že po kliknutí chceme celé okno znova skryť:

```
script.Parent.MouseButton1Click:Connect(function()
    -- nami naprogramované správanie
    print("Áno, ja som tiež!")

    script.Parent.Parent.Visible = false   -- schová celý Frame
end)
```

20.1 Úloha – NPC postava má otázku

Pridaj do hry NPC postavu. Vytvor interaktívne okno s otázkou, z akej si obce.

Na výber budú dve možnosti – `Áno` a `Nie`. Okno sa zobrazí po kliknutí myšou na NPC postavu.

Po kliknutí na tlačidlo sa okno zatvorí a do konzoly sa vypíše odpoveď, napríklad `áno, ja som tiež` alebo `nie, odtiaľ nie som`.

20.2 Bonus – Odmena za odpoveď

Rozšír úlohu: ak hráč odpovie `Áno`, zafarbi mu nejaký objekt v hre na zeleno alebo mu pridaj body do premennej.

Čo si sa naučil

- NPC sa pridáva cez **Avatar → Rig Builder**.
- `Frame` zoskupuje prvky GUI, aby sa dali skryť naraz.
- `GetMouse().Target` zistí, na čo hráč klikol – vždy over, či to nie je `nil`.

21. Lekcia – Nástroje (Tools)

Čo je nástroj

Nástroje sa líšia od ostatných objektov tým, že keď cez nástroj hráč prejde, zmizne zo sveta a objaví sa v **inventári**. Ak má hráč v inventári viac vecí, môže medzi nimi prepínať stlačením čísel 1 – 9.

Pridanie nástroja z Toolboxu

V sekcii **Home** klikni na **Toolbox**. Otvorí sa okno, v ktorom vieš pridávať rôzne predmety. Do vyhľadávacieho poľa napíš **grapple hook**, s ktorým budeme pracovať.

Po kliknutí naň sa nástroj vloží do **StarterPacku**. To znamená, že po spustení hry ho hráč bude mať v inventári. Ak by sme ho vložili do **Workspace**, objavil by sa v hre ako predmet na zdvihnutie.

Pridanie nástroja programom

Niekedy chceme dať hráčovi nástroj do inventára až počas hry. V takom prípade daj nástroj do **ReplicatedStorage** a použi tento kód:

```
local tool = game:GetService("ReplicatedStorage").GrappleHook:Clone()
local hrac = game:GetService("Players").LocalPlayer

tool.Parent = hrac:FindFirstChildOfClass("Backpack")
```

Ako skontrolovať, či sa nástroj už v inventári nachádza:

```
if hrac.Backpack:FindFirstChild("GrappleHook") then
    -- tento kód sa vykoná, ak je GrappleHook v inventári
end
```

Vytvorenie vlastného nástroja

V Exploreri choď myšou na **Workspace** a klikni na „+“
Vyber **Tool**
Choď myšou na pridaný Tool, klikni na „+“ a vyber **Part**
Premenuj tento Part na **Handle**
Spusti hru a skús nástroj zobrať – po prejdení postavou by si ho mal mať v ruke alebo v inventári

Handle musí mať presne toto meno

Part vnútri nástroja sa **musí** volať `Handle` (s veľkým H). Je to meno, ktoré Roblox hľadá.

Ak sa bude volať inak, nástroj sa nedá zobrať do ruky a nebudeš vedieť prečo.

21.1 Úloha – Grapple hooky šup do inventára

Pridaj do hry Part a vytvaruj ho do platformy, na ktorú sa dá stúpiť.

Naprogramuj, aby sa po dotknutí ľavou nohou tejto platformy pridal do inventára grapple hook.

21.2 Úloha – Organizovaný inventár

Uprav predošlú úlohu tak, aby sa grapple hook pridal **iba vtedy, ak tam ešte nie je**.

21.3 Úloha – Zobrazenie informácie o predmete

Vytvor `TextLabel`, na ktorom bude aktuálna informácia, či máme grapple hook v inventári alebo nie.

Čo si sa naučil

- Nástroj (`Tool`) sa po zdvihnutí objaví v inventári.
- `StarterPack` = hráč to má hneď, `ReplicatedStorage` = dáme mu to programom.
- Part vnútri nástroja sa musí volať `Handle`.

22. Lekcia – Udalosti Equipped a Unequipped

Použitie udalostí

Keď sa prepneme na nástroj v inventári, nastane jeho udalosť **`Equipped`** (vzal som ho do ruky). Opätovným stlačením toho istého čísla nastane **`Unequipped`** (odložil som ho). Stlačením iného čísla nastane `Unequipped` pre starý nástroj a `Equipped` pre nový.

```
local function onEquip()
    -- čo sa má stať, keď vezmeme nástroj do ruky
end
script.Parent.Equipped:Connect(onEquip)

local function onUnequip()
    -- čo sa má stať, keď nástroj odložíme
end
script.Parent.Unequipped:Connect(onUnequip)
```

Pozor na preklep

Všimni si, že druhá udalosť je **`Unequipped`**, nie `Equipped`.

Ak by si omylom pripojil obe funkcie k `Equipped`, obe by sa spustili naraz pri zdvihnutí nástroja a pri odložení by sa nestalo nič. Chyba by sa hľadala veľmi ťažko, lebo Roblox by nenahlásil žiadnu chybu – kód by bol správny, len by robil niečo iné.

22.1 Úloha – Získaj grapple hook

K dispozícii máš predpripravené prostredie. Našou úlohou je pridať doň interaktivitu.

Do `grapplePlosiny` pridaj skript, v ktorom sa po dotknutí ľavou nohou rodičovského objektu dá do inventára grapple hook. Ošetri aj podmienku, či ho už v inventári nemáme (maximálne 2 kusy).

Následne vytvor `Tool`, ktorý bude malá žltá guľa, a nazvi ho `platformUnlocker`. V tomto Toole naprogramuj logiku: keď máme guličku v ruke (Equipped), zobrazí sa skrytá stena, na ktorej nás čaká ďalšia úloha. Ak ju v ruke nemáme, stena zostane neviditeľná.

22.2 Úloha – Nájdi a označ správnu vežu

Pokračujeme v budovaní hry. Využijeme druhý predmet, ktorý už v hre je – `hint`, ktorý nám rozsvieti správnu vežu, kam sa pomocou grapple hooku dostaneme.

Do objektu `light` pridaj cez „+“ **PointLight**. Následne v Toole `hint` vytvor skript a naprogramuj logiku: keď máme `hint` v ruke (Equipped), rozsvieti sa správna veža (objekt `light`), inak nesvieti.

Ako poslednú podúlohu naprogramuj v objekte `light` správanie, že keď sa ho dotkneme (dostali sme sa na správnu vežu), do výstupu sa vypíše text `Vyhral si`.

Čo si sa naučil

- `Equipped` = nástroj v ruke, `Unequipped` = odložený.
- Obe udalosti sa pripájajú cez `:Connect()`.

23. Lekcia – Animácie

Vytvorenie animácie

Najprv pridaj postavu, na ktorej animáciu vytvoríš. Použi rovnaký postup ako v 20. lekcii (Avatar → Rig Builder).
Otvor **Animation Editor**, ktorý sa nachádza v sekcii **Avatar**.
Klikni na postavu.
Zobrazí sa okno, kde zadáš názov animácie (napr. útok, mávanie, beh) a klikneš na modré tlačidlo **Create**.

Časti tela postavy popresúvaj a pootáčaj tak, ako chceš, aby boli v **počiatočnom bode** animácie.
Posuň modrú rysku napravo, napríklad na 0:07.
Znova pootáčaj a popresúvaj časti tela tak, ako chceš, aby boli v čase 0:07.
Počítač automaticky doplní všetky body medzi tými, ktoré si zadal, aby bola animácia plynulá.
Animáciu si vieš prehrať kliknutím na **play** vľavo v Animation Editore.
Môžeš pridať viac bodov, alebo jednotlivé body posúvať a mazať.

Nastav prioritu animácie na **action**, aby mala vyššiu prioritu ako napr. pohyb. Klikni na „…“ vedľa názvu animácie a choď na **Set animation priority → action**.
Keď je animácia hotová, znova klikni na „…“ a zvoľ **Publish to Roblox**.
Vyplň formulár a klikni na modré tlačidlo **Submit**.
Skopíruj si **ID animácie** – budeme ho potrebovať.

> **Stratil si ID animácie?**
>
> Netreba sa zľaknúť, dá sa dohľadať. Choď do sekcie **View** a klikni na **Toolbox**.
>
> V Toolboxe choď do sekcie **Inventory** (druhá zľava) a nastav **My Animations**. Zobrazia sa všetky tvoje animácie.
>
> Klikni na tú svoju **pravým tlačidlom** myši a zvoľ **Copy Asset ID**.

Pridanie animácie do hry

Choď myšou na objekt v okne Explorer, do ktorého chceš animáciu pridať (my ju budeme pridávať pod skript). Napravo od názvu sa objaví „+“ – klikni naň a zo zoznamu vyber **Animation**.

Klikni na pridanú animáciu a nastav jej vlastnosť **`AnimationId`** na ID tvojej animácie. Skopírovanú hodnotu vlož do kolónky (Ctrl + V). Roblox Studio k nej automaticky pridá `rbxassetid://`.

> **rbxassetid, nie rbassetid**
>
> Správny tvar je **`rbxassetid://`** – s písmenom **x** po `rb`.
>
> Ak tam bude `rbassetid://`, animácia sa nenačíta a v Outpute uvidíš chybu o neplatnom asset ID.

Programovanie prehratia animácie

Na to, aby sme animáciu prehrali, musíme určiť, na ktorej postave sa bude prehrávať (hráč, NPC) a ktorá animácia to bude.

```
local animation = script.Animation           -- načítanie animácie
local humanoid = script.Parent.Humanoid      -- načítanie NPC postavy

local animationTrack = humanoid.Animator:LoadAnimation(animation)

animationTrack:Play()             -- spustenie animácie
animationTrack:AdjustSpeed(2)     -- nastavenie rýchlosti animácie
```

Animator, nie Humanoid

V starších návodoch uvidíš `humanoid:LoadAnimation(animation)`. Roblox tento spôsob **zrušil**.

Správne je dnes **`humanoid.Animator:LoadAnimation(animation)`**. Ak použiješ starý zápis, dostaneš varovanie a časom prestane fungovať úplne.

Podobne vieme prehrať animáciu aj na našej postave. V tom prípade treba do **StarterPlayer → StarterCharacterScripts** pridať **LocalScript**, doňho **Animation** a použiť rovnaký kód.

ProximityPrompt a animácie

Do objektu vieme cez „+“ pridať **ProximityPrompt**. Vďaka nemu sa v hre objaví možnosť interagovať s objektom, čo vieme spojiť s prehratím animácie.

```
local prompt = script.Parent.ProximityPrompt
local animation = script:WaitForChild("Animation")

prompt.Triggered:Connect(function(player)
    -- kód na načítanie Humanoida a animácie
    -- ďalší kód pre spustenie, zastavenie animácie atď.
end)
```

23.1 Úloha – Spustenie vlastnej animácie na NPC postave

Pridaj do hry NPC postavu a vytvor vlastnú animáciu. Nezabudni na správne nastavenie `AnimationId`.

V skripte v NPC postave naprogramuj prehratie animácie. Po 5 sekundách zrýchli prehrávanie 2-krát. Po ďalších 5 sekundách nastav rýchlosť na 0.3. Pozoruj zmeny.

23.2 Úloha – Animácia na našej postave

Na vlastnej postave (`StarterCharacterScripts`) pridaj skript, ktorým zapneš animáciu. Použi `AnimationId`: **616158929**.

Náš hráč by mal mať pri pohybe iný postoj v porovnaní s normálnym pohybom bez animácie.

23.3 Úloha – Animácia po interakcii s objektom

Do objektu pridaj **ProximityPrompt**. V skripte naprogramuj správanie tak, že keď hráč interaguje s objektom, prehrá sa animácia s `AnimationId`: **3716468774**.

Čo si sa naučil

- Animácia sa vytvára v **Animation Editore** a musí sa publikovať, aby získala `AnimationId`.
- Správny tvar je `rbxassetid://`.
- Načítava sa cez `humanoid.Animator:LoadAnimation()`.

Príloha A – Ťahák

Základné príkazy

```
print("text")                  -- vypíše na výstup
print(cislo)                   -- vypíše hodnotu premennej
task.wait(2)                   -- počká 2 sekundy
-- komentár                    -- poznámka, počítač ju ignoruje
```

Premenné

```
local penize = 100             -- vytvorenie (local len raz!)
penize = 35                    -- zmena hodnoty
penize = penize + 1            -- zvýšenie o 1
penize += 1                    -- to isté, kratšie
local text = "ahoj"            -- text v ROVNÝCH úvodzovkách
local pravda = true            -- logická hodnota
print("Mám " .. penize .. " eur")   -- spájanie textu dvoma bodkami
```

Vlastnosti objektov

```
script.Parent.Anchored = true          -- objekt, ku ktorému patrí skript
workspace.Kvader.Anchored = true       -- objekt podľa mena

workspace.Kvader.Color = Color3.fromRGB(255, 0, 0)
workspace.Kvader.Material = Enum.Material.Neon
workspace.Kvader.Transparency = 1
workspace.Kvader.CanCollide = false
workspace.Kvader.Position = Vector3.new(10, 5, 0)
workspace.Kvader.Shape = Enum.PartType.Ball
```

Podmienky

```
if penize >= cena then
    print("Kupujeme!")
elseif penize > 0 then
    print("Máme málo.")
else
    print("Nemáme nič.")
end

-- ==  ~=  <  >  <=  >=          and   or   not
```

Funkcie

```
local function scitaj(a, b)
    return a + b
end

local vysledok = scitaj(3, 5)   -- zavolanie funkcie
```

Cykly

```
for i = 1, 5 do                 -- zopakuje 5-krát
    print(i)
    task.wait(1)
end

while pocet < 5 do              -- opakuje, kým platí podmienka
    pocet += 1
    task.wait(1)
end
```

Udalosti

```
script.Parent.Touched:Connect(function(hit)
    if hit.Name == "LeftFoot" then
    end
end)

script.Parent.MouseButton1Click:Connect(function()
end)

local UIS = game:GetService("UserInputService")
UIS.InputBegan:Connect(function(input)
    if input.KeyCode == Enum.KeyCode.Q then
    end
end)
```

Užitočné materiály a farby

Materiály	**Farby cez fromRGB**
`Neon`, `Brick`, `Rock`, `Asphalt`	červená: `255, 0, 0`
`Grass`, `Sand`, `Wood`, `Metal`	zelená: `0, 255, 0`
`Ice`, `Glass`, `CrackedLava`	modrá: `0, 0, 255`
`Plastic`, `SmoothPlastic`	žltá: `255, 255, 0`

Príloha B – Časté chyby a ako ich odstrániť

Chybové hlášky

Hláška v Outpute	Čo znamená	Čo skontrolovať
`attempt to index nil with '...'`	Roblox objekt nenašiel	Preklep v mene objektu; veľké/malé písmená; existuje objekt v Exploreri?
`Expected 'end'`	Chýba ukončenie bloku	Spočítaj `end` – každé `if`, `for`, `while`, `function` musí mať jedno
`Unfinished string`	Neuzavretá úvodzovka	Chýba `"` na konci textu
`'=' expected near ...`	Chyba v zápise	Nemáš `=` tam, kde má byť `==`?
`attempt to perform arithmetic`	Počítaš s textom	Nemáš číslo omylom v úvodzovkách?
`is not a valid member of`	Taká vlastnosť neexistuje	Preklep vo vlastnosti; `Transparency` nie `transparency`

Nič sa nedeje a v Outpute nie je chyba

- **Je skript na správnom mieste?** Skripty na objekty patria do Workspace, skripty na obrazovku a ovládanie do StarterGui / StarterPlayerScripts.
- **Je to Script alebo LocalScript?** Obrazovku a klávesnicu obsluhuje LocalScript, objekty vo svete obyčajný Script.
- **Spustil si hru?** Skript beží až po stlačení Play.
- **Nemenil si to počas behu hry?** Zmeny počas Play sa neuložia.
- Vlož na začiatok skriptu `print("som tu")` a over, či sa skript vôbec spustil.

Objekt padá alebo sa nedá naň stúpiť

- Objekt padá → nastav mu `Anchored = true`
- Prepadávaš cez objekt → skontroluj `CanCollide` (musí byť `true`)
- Objekt nevidno → skontroluj `Transparency` (má byť 0)

Hra zamrzla a nedá sa vypnúť

Skoro určite máš nekonečný `while true do` cyklus **bez `task.wait()`**. Roblox Studio bude treba zavrieť cez Správcu úloh. Nabudúce nezabudni na čakanie.

Zvuk sa neprehrá

- Väčšina zvukov v Toolboxe je súkromná a nefunguje – skús iný alebo použi zoznam od učiteľa.
- Je zvuk v `Workspace` a má správne meno v skripte?
- Nie je `Volume` na nule?